Lab 10 – Binding, INotifyPropertyChanged, ObservableCollection

Title: Data Binding and Automatic UI Updates

Objectives

This was a fundamental shift in how the application works. The goal was to move away from manual code-behind updates and implement true data binding. This involves implementing INotifyPropertyChanged to automatically notify the UI when data changes, and using ObservableCollection<T> for automatic collection updates. This makes the UI and data layers completely decoupled.

Implementation

Updated Angajat Class with INotifyPropertyChanged

```
using System;
using System.ComponentModel;
using System.Runtime.CompilerServices;

namespace SalariiModel
{
   public class Angajat : INotifyPropertyChanged
   {
      private int _id;
      private string _nume;
      private string _prenume;
      private string _functie;
```

```
private Departament _departament;
private TipSalariu _tipSalariu;
private double _salariuBrut;
private double _tarifOra;
private int _oreLucrate;
private double _bonusuri;
private DateTime _dataAngajarii;

public int Id
{
   get => _id;
   set { _id = value; OnPropertyChanged(); }
}

public string Nume
{
   get => _nume;
   set { _nume = value; OnPropertyChanged(); }
}

public string Prenume
{
   get => _prenume;
   set { _prenume = value; OnPropertyChanged(); }
}

public string Functie
{
   get => _functie;
   set { _functie = value; OnPropertyChanged(); }
```

```
}

public Departament Departament
{
   get => _departament;
   set { _departament = value; OnPropertyChanged(); }
}

public TipSalariu TipSalariu
{
   get => _tipSalariu;
   set { _tipSalariu = value; OnPropertyChanged(); }
}

public double SalariuBrut
{
   get => _salariuBrut;
   set { _salariuBrut = value; OnPropertyChanged(); }
}

public double TarifOra
{
   get => _tarifOra;
   set { _tarifOra = value; OnPropertyChanged(); }
}

public int OreLucrate
{
   get => _oreLucrate;
   set { _oreLucrate = value; OnPropertyChanged(); }
```

```
}

public double Bonusuri
{
   get => _bonusuri;
   set { _bonusuri = value; OnPropertyChanged(); }
}

public DateTime DataAngajarii
{
   get => _dataAngajarii;
   set { _dataAngajarii = value; OnPropertyChanged(); }
}

// Computed properties with INotifyPropertyChanged
private double _salariuBrutEfectiv;
public double SalariuBrutEfectiv
{
   get
   {
      if (TipSalariu == TipSalariu.Orar)
         return TarifOra * OreLucrate + Bonusuri;
      else
         return SalariuBrut + Bonusuri;
   }
}

private double _salariuNet;
public double SalariuNet => SalariuBrutEfectiv * 0.75;
```

```
    public string NumeComplet => $"{Nume} {Prenume}";

    public Angajat()
    {
        DataAngajarii = DateTime.Today;
    }

    // INotifyPropertyChanged Implementation
    public event PropertyChangedEventHandler PropertyChanged;

    protected void OnPropertyChanged([CallerMemberName] string propertyName = null)
    {
        PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(propertyName));

        // Notify changes on dependent properties
        if (propertyName == nameof(TipSalariu) ||
            propertyName == nameof(TarifOra) ||
            propertyName == nameof(OreLucrate) ||
            propertyName == nameof(SalariuBrut) ||
            propertyName == nameof(Bonusuri))
        {
            PropertyChanged?.Invoke(this, new
PropertyChangedEventArgs(nameof(SalariuBrutEfectiv)));
            PropertyChanged?.Invoke(this, new
PropertyChangedEventArgs(nameof(SalariuNet)));
        }

        if (propertyName == nameof(Nume) || propertyName == nameof(Prenume))
        {
            PropertyChanged?.Invoke(this, new
PropertyChangedEventArgs(nameof(NumeComplet)));
```

```
        }
    }
  }
}
```

Updated Storage with ObservableCollection

```
using System.Collections.ObjectModel;
using System.Linq;

namespace SalariiStocare
{
  public class AdministrareAngajatiMemorie : IStocareData
  {
    private ObservableCollection<Angajat> _angajati = new
ObservableCollection<Angajat>();
    private int _nextId = 1;

    public void AddAngajat(Angajat angajat)
    {
      angajat.Id = _nextId++;
      _angajati.Add(angajat);
    }

    public ObservableCollection<Angajat> GetAngajati()
    {
      return _angajati;
    }

    // ... other methods unchanged ...
  }
```

}

MainWindow.xaml with Data Binding

```
<Window x:Class="SalariiUI.MainWindow"
      xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
      xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
      xmlns:model="clr-namespace:SalariiModel;assembly=SalariiModel"
      Title="Sistem Salarii - Binding Demo"
      Height="700"
      Width="900"
      WindowStartupLocation="CenterScreen">

    <Window.Resources>
      <!-- ... resources ... -->
    </Window.Resources>

    <Grid Margin="15">
      <Grid.RowDefinitions>
        <RowDefinition Height="*"/>
        <RowDefinition Height="Auto"/>
      </Grid.RowDefinitions>

      <Grid Grid.Row="0">
        <Grid.ColumnDefinitions>
          <ColumnDefinition Width="*"/>
          <ColumnDefinition Width="Auto"/>
          <ColumnDefinition Width="*"/>
        </Grid.ColumnDefinitions>

        <!-- Employee List (Left) -->
```

```
        <GroupBox Header="Lista Angajați" Grid.Column="0">
          <Grid>
            <Grid.RowDefinitions>
              <RowDefinition Height="*"/>
              <RowDefinition Height="Auto"/>
            </Grid.RowDefinitions>

            <ListBox Grid.Row="0" x:Name="lstAngajati"
                 ItemsSource="{Binding Angajati}"
                 SelectedItem="{Binding AngajatSelectat, Mode=TwoWay}"
                 DisplayMemberPath="NumeComplet"/>

            <StackPanel Grid.Row="1" Orientation="Horizontal" HorizontalAlignment="Center"
Margin="0,10">
              <Button Content="Adaugă" Command="{Binding AdaugaCommand}"
                   Background="{StaticResource AccentBrush}" Foreground="White"/>
              <Button Content="Șterge" Command="{Binding StergeCommand}"
                   Background="{StaticResource DangerBrush}" Foreground="White"/>
            </StackPanel>
          </Grid>
        </GroupBox>

        <!-- Separator -->
        <GridSplitter Grid.Column="1" Width="5" HorizontalAlignment="Center"
VerticalAlignment="Stretch"/>

        <!-- Employee Details (Right) -->
        <GroupBox Header="Detalii Angajat" Grid.Column="2">
          <StackPanel Margin="10">
            <StackPanel Orientation="Horizontal">
              <Label Content="Nume:" Width="80"/>
```

```
            <TextBox Text="{Binding AngajatSelectat.Nume, Mode=TwoWay,
UpdateSourceTrigger=PropertyChanged}"
                     Width="150" Margin="5,0"/>
        </StackPanel>

        <StackPanel Orientation="Horizontal">
            <Label Content="Prenume:" Width="80"/>
            <TextBox Text="{Binding AngajatSelectat.Prenume, Mode=TwoWay,
UpdateSourceTrigger=PropertyChanged}"
                     Width="150" Margin="5,0"/>
        </StackPanel>

        <StackPanel Orientation="Horizontal">
            <Label Content="Funcție:" Width="80"/>
            <TextBox Text="{Binding AngajatSelectat.Functie, Mode=TwoWay,
UpdateSourceTrigger=PropertyChanged}"
                     Width="150" Margin="5,0"/>
        </StackPanel>

        <StackPanel Orientation="Horizontal">
            <Label Content="Salariu Brut:" Width="80"/>
            <TextBox Text="{Binding AngajatSelectat.SalariuBrut, Mode=TwoWay,
UpdateSourceTrigger=PropertyChanged}"
                     Width="150" Margin="5,0"/>
        </StackPanel>

        <Separator Margin="0,10"/>

        <StackPanel Orientation="Horizontal" Background="#F0F0F0" Padding="10">
            <Label Content="Salariu Net:" FontWeight="Bold" Width="80"/>
            <Label Content="{Binding AngajatSelectat.SalariuNet, StringFormat={}{0:C}}"
                   FontWeight="Bold" Foreground="{StaticResource AccentBrush}"/>
```

```
            </StackPanel>
          </StackPanel>
        </GroupBox>
      </Grid>

      <StatusBar Grid.Row="1">
        <StatusBarItem>
          <TextBlock Text="{Binding Angajati.Count, StringFormat='Total: {0} angajați'}"/>
        </StatusBarItem>
      </StatusBar>
    </Grid>
</Window>
```

ViewModel with Commands

```
using SalariiModel;
using SalariiStocare;
using SalariiUI.Commands;
using System.Collections.ObjectModel;
using System.ComponentModel;
using System.Runtime.CompilerServices;
using System.Windows.Input;

namespace SalariiUI.ViewModels
{
    public class MainViewModel : INotifyPropertyChanged
    {
        private readonly IStocareData _admin;
        private ObservableCollection<Angajat> _angajati;
        private Angajat _angajatSelectat;
```

```
    public ObservableCollection<Angajat> Angajati
    {
        get => _angajati;
        set { _angajati = value; OnPropertyChanged(); }
    }

    public Angajat AngajatSelectat
    {
        get => _angajatSelectat;
        set { _angajatSelectat = value; OnPropertyChanged(); }
    }

    public ICommand AdaugaCommand { get; }
    public ICommand StergeCommand { get; }

    public MainViewModel()
    {
        _admin = StocareFactory.GetAdministratorStocare();
        Angajati = new ObservableCollection<Angajat>(_admin.GetAngajati());

        AdaugaCommand = new RelayCommand(_ => AdaugaAngajat());
        StergeCommand = new RelayCommand(_ => StergeAngajat(), _ => AngajatSelectat !=
null);
    }

    private void AdaugaAngajat()
    {
        var angajat = new Angajat
        {
            Nume = "Nou",
            Prenume = "Angajat",
```

```
            Functie = "Stagiar",
            SalariuBrut = 3000
        };
        _admin.AddAngajat(angajat);
        Angajati.Add(angajat);
        AngajatSelectat = angajat;
    }

    private void StergeAngajat()
    {
        if (AngajatSelectat == null) return;
        _admin.StergeAngajat(AngajatSelectat.Id);
        Angajati.Remove(AngajatSelectat);
        AngajatSelectat = null;
    }

    public event PropertyChangedEventHandler PropertyChanged;
    protected void OnPropertyChanged([CallerMemberName] string name = null)
        => PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(name));
  }
}
```

RelayCommand Implementation

```
using System;
using System.Windows.Input;

namespace SalariiUI.Commands
{
  public class RelayCommand : ICommand
  {
```

```
    private readonly Action<object> _execute;
    private readonly Predicate<object> _canExecute;

    public RelayCommand(Action<object> execute, Predicate<object> canExecute = null)
    {
      _execute = execute ?? throw new ArgumentNullException(nameof(execute));
      _canExecute = canExecute;
    }

    public bool CanExecute(object parameter) => _canExecute?.Invoke(parameter) ?? true;
    public void Execute(object parameter) => _execute(parameter);

    public event EventHandler CanExecuteChanged
    {
      add { CommandManager.RequerySuggested += value; }
      remove { CommandManager.RequerySuggested -= value; }
    }
  }
}
```

Why I Used This Approach

INotifyPropertyChanged: This interface is essential for WPF data binding. When a property changes, the UI is automatically notified and updates. This eliminates manual UI updates in code-behind.

CallerMemberName Attribute: This automatically provides the property name to `OnPropertyChanged`, reducing errors from hardcoded strings.

Dependent Property Notification: When `TipSalariu` changes, I also notify that `SalariuBrutEfectiv` and `SalariuNet` may have changed. This ensures computed properties stay in sync with the UI.

ObservableCollection: This collection automatically notifies the UI when items are added, removed, or changed. The DataGrid updates automatically without any manual reloading.

Two-Way Binding: `Mode=TwoWay` means changes in the UI update the data and vice versa. `UpdateSourceTrigger=PropertyChanged` updates the data source as the user types, not just when focus is lost.

Commands: Using `ICommand` implementations (`RelayCommand`) decouples user actions from UI events. Commands can be bound to buttons and other controls.

ViewModel Pattern: The ViewModel holds the data and commands, while the View (XAML) only handles the UI. This separation of concerns makes the code more testable and maintainable.

GridSplitter: Allows the user to resize the left and right panels, improving usability.

Testing & Results

I tested the data binding by:

1. Selecting an employee from the ListBox – the details appeared in the right panel
2. Typing in the TextBoxes – the data updated immediately (due to `PropertyChanged` trigger)
3. Clicking "Adaugă" – a new employee appeared in the list automatically
4. Clicking "Șterge" – the selected employee was removed from the list automatically

The UI updated seamlessly without any manual code-behind intervention.

Reflection

The MVVM pattern (Model-View-ViewModel) is what we're moving toward, and data binding is the core of it. The code is now much cleaner – I removed all the manual `txtName.Text = employee.Name` statements. The UI is truly decoupled from the data, which makes testing easier and reduces bugs. The `INotifyPropertyChanged` pattern is essential for WPF development and understanding it is crucial. `ObservableCollection` makes working with collections in a data-bound context much simpler.